

Delivery Truck Pallet Packing Optimization

Design of Algorithms 2024/2025

2ndyear, 2ndsemester, L.EIC, FEUP

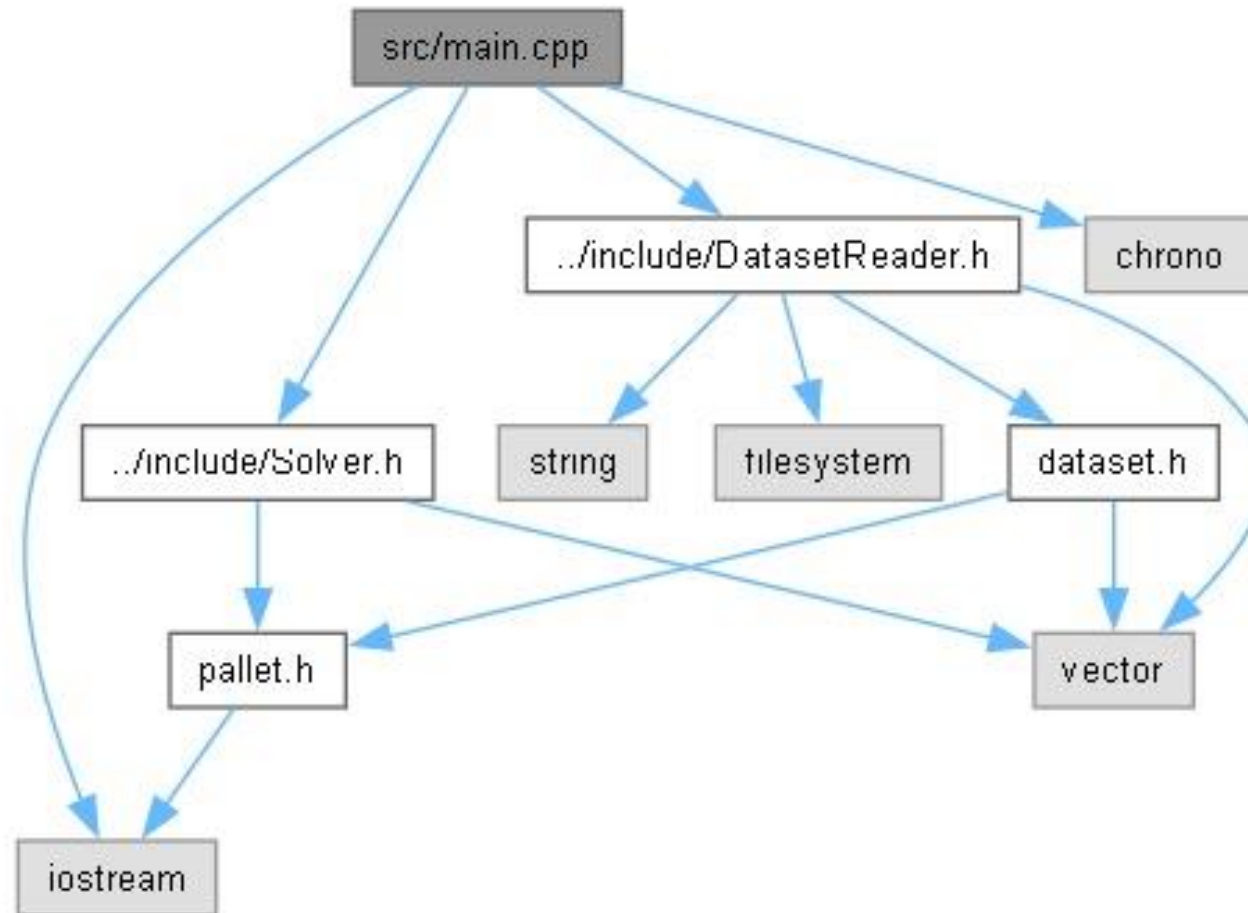
Class 15, Group 2:

Afonso Caetano, 202303614

Gonalo Ferreira, 202304562

Joo Pedro Sousa, 202304607

Class Diagram



Reading the Dataset

➤ The provided dataset consists of **two CSV files**:

- A **truck file**, which contains the truck's capacity and the maximum number of pallets;

1	Capacity,Pallets
2	524,33

→ Example of a TruckAndPallets XX.csv File

- A **pallets file**, which contains information about each pallet;

1	Pallet,Weight,Profit
2	1,70,10
3	2,60,5
4	3,50,10
5	4,33,8
6	5,33,9
7	6,33,7
8	7,11,7
9	8,7,2
10	9,3,3

→ Example of a Pallets XX.csv File

Reading the Dataset

1. TruckAndPallets_XX.csv File: Reading the Truck Data

- ✓ To facilitate reading, the structure dataset is created, which holds all the necessary data;
- ✓ The truck file is opened using an input file stream;
- ✓ The second line contains two values: capacity and number of pallets;
- ✓ These values are extracted from the line using a string stream and stored in the dataset object;

2. Pallets File: Reading the Pallet Data

- ✓ A vector<pallet> is created inside the dataset structure to store each pallet;
- ✓ Each line of the file contains three values: ID, weight, and profit of a pallet;
- ✓ Each line is parsed, and a pallet object is created and filled with the data;
- ✓ That object is added to the pallets vector inside the dataset;
- ✓ Finally, the function returns a fully filled dataset object with all the truck and pallet data;

Algorithms and Implementations

➤ Backtracking Algorithm

- Solves the 0/1 Knapsack Problem using exhaustive search via recursion and backtracking.
- **Description**
 - At each step, the algorithm chooses to include or exclude the current item.
 - Recursion continues until the end of the item list or the knapsack capacity is exceeded.
 - If a branch exceeds the capacity, the algorithm backtracks to explore other paths.
 - Keeps track of the best valid solution found during the search.
- **Data Structures Used:** vector<pallet> – holds all pallets; vector<int> – current and best combination of pallet IDs.
- **Complexities:** Time: $O(2^n)$; Space: $O(n)$
- **Advantages:** Guarantees the optimal solution.
- **Disadvantages:** Exponential time complexity → not feasible for large datasets.

Algorithms and Implementations

```
void backtrackingKnapsack(const std::vector<pallet>& pallets, int maxWeight);
```



backtrackingKnapsack initializes the process and tracks the best solution.

```
void backtrack(const std::vector<pallet>& pallets, int index, int currentWeight, int currentProfit, int maxWeight, std::vector<int>& currentCombination);
```



Helper function backtrack which performs the recursive search:

- At each step, it branches into:
 - Including the current pallet (if weight allows);
 - Excluding the current pallet;
- When reaching the end (`index == pallets.size()`), it:
 - Verifies if the solution is valid ($\text{weight} \leq \text{maxWeight}$);
 - Updates the global bestProfit, bestWeight, and bestCombination if better;

Algorithms and Implementations

➤ Dynamic Programming – Bottom-Up Approach

- Solves the 0/1 Knapsack Problem using a bottom-up dynamic programming strategy.

```
void dynamicProgrammingKnapsack(const std::vector<pallet>& pallets, int maxWeight);
```



Builds a 2D DP table $dp[i][w]$, where:

- i is the index of the pallet;
- w is the weight capacity (from 0 to maxWeight);
- $dp[i][w]$ stores the maximum profit achievable using the first i pallets and total weight $\leq w$
- The Table is filled iteratively: For each pallet, we decide whether to include or exclude it based on weight and profit. After filling the table, a backtracking phase reconstructs the optimal combination of pallet IDs
- **Complexities:** Time: $O(n * W)$; Space: $O(n * W)$
- **Advantages:** Guarantees the optimal solution; Avoids redundant computations by storing intermediate results in a DP table

Algorithms and Implementations

➤ Greedy Algorithm – Profit-to-Weight Approach

- Solves the 0/1 Knapsack Problem approximately using a greedy heuristic based on the profit-to-weight ratio.

```
void greedyKnapsack(const vector<pallet>& pallets, int maxWeight)
```



- Sorts the pallets in descending order of profit-to-weight ratio: Ratio = Profit/Weight
 - Iteratively adds the most "profitable per unit weight" pallet to the knapsack, as long as it fits within the remaining capacity.
 - Stops when the next pallet doesn't fit or all are processed
-
- **Complexities:** Time : $O(n \log n)$; Space : $O(n)$
 - **Advantages:** Very fast compared to exhaustive approaches; Simple and intuitive.
 - **Disadvantages:** May not yield optimal results for 0/1 Knapsack (unlike fractional version).

Algorithms and Implementations

➤ Greedy + Local Search Algorithm

- Enhances the Greedy solution by applying Local Search to escape suboptimal configurations.

```
void greedyPlusLocalSearchKnapsack(const vector<pallet>& pallets, int maxWeight)
```



- Starts from the Greedy solution;
 - Then applies Local Search:
 - Iteratively attempts to swap pallets in the solution with those not included
 - Only accepts changes that improve the total profit and respect the weight constraint
 - Greedy gives a fast, decent starting point
 - Local Search helps overcome its limitations : Fixes cases where Greedy approach skips good global combinations due to local decisions.
-
- **Complexities:** Time : $O(n^2)$; Space : $O(n)$
 - **Advantages:** Very fast compared to exhaustive approaches; More accurate than pure Greedy

Algorithms and Implementations

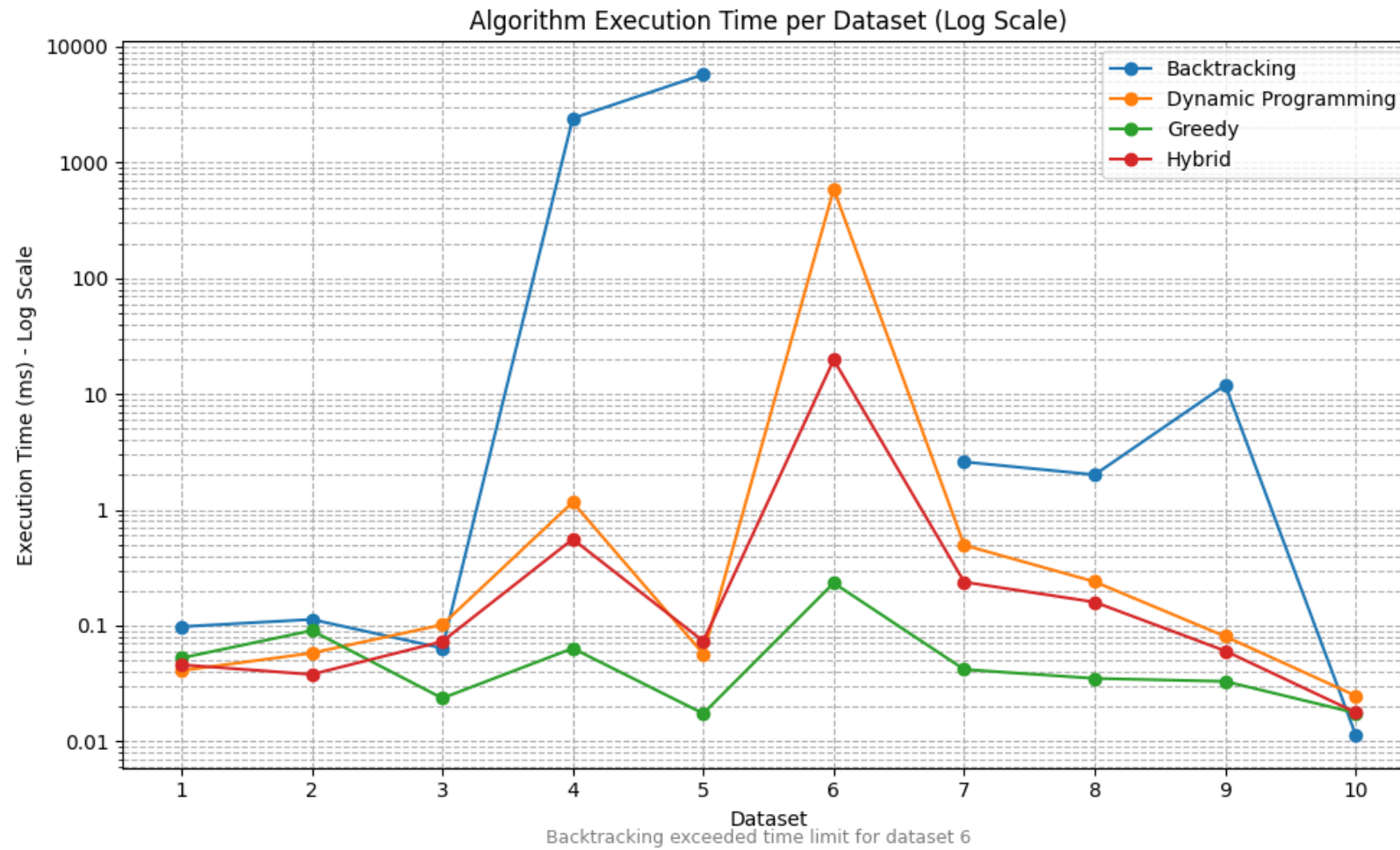
➤ Hybrid Approach, (Greedy + Local Search) + Dynamic Programming

```
void hybridKnapsack(const std::vector<pallet>& pallets, int maxWeight)
```

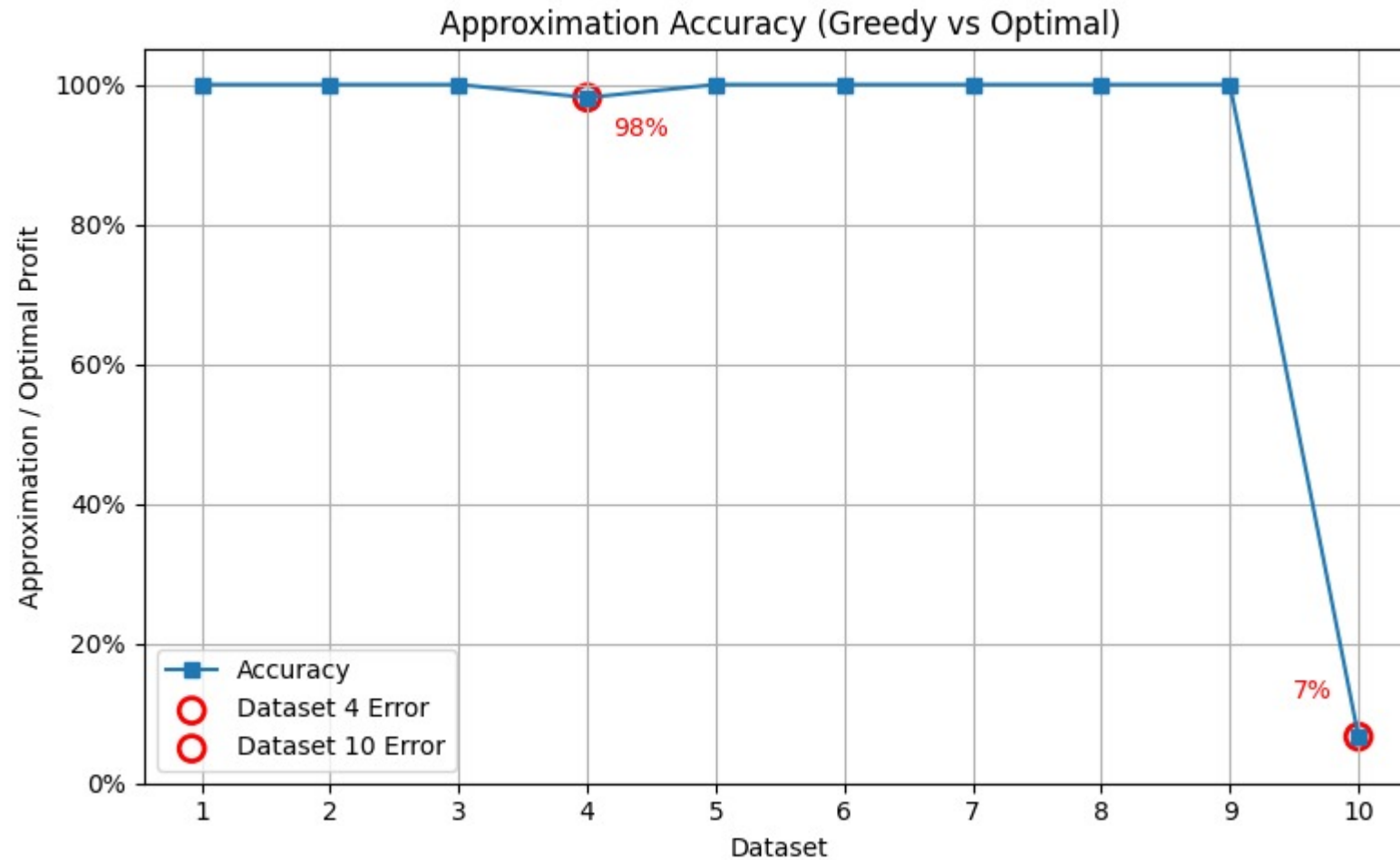


- Designed for large instances where exhaustive search (e.g., backtracking) or full DP is too slow.
 - Starts by generating a Greedy solution.
 - Then applies Local Search.
 - Dynamic Programming may be applied on small subgroups to refine local improvements.
- ➔ Greedy + Local Search Algorithm
-
- **Complexities:** Time : **Greedy** - $O(n \log n)$, **Local Search** - $O(n^2)$, **Hybrid Overall** - $O(n^2)$ faster in practice;
Space : $O(n)$
 - **Advantages:** Scales well with larger datasets; Produces better results than Greedy alone; Much faster than full DP or Backtracking on big inputs.
 - **Disadvantages:** Final result depends on initial Greedy solution quality.

Algorithms and Implementations



Algorithms and Implementations



User Interface

```
=== Pallet Packing Optimization ===  
1. Select and use a dataset  
2. Exit
```

```
=== Datasets Disponiveis ===  
Dataset 01  
Dataset 02  
Dataset 03  
Dataset 04  
Dataset 05  
Dataset 06  
Dataset 07  
Dataset 08  
Dataset 09  
Dataset 10  
  
Escolhe o numero do dataset (ex: 01):
```

User Interaction Flow

```
Best combination (Backtracking):  
Pallets: 1 3 7 15  
Total weight: 15  
Total profit: 15  
Execution time: 123.654 ms
```

```
Escolhe o numero do dataset (ex: 01):05  
  
Truck capacity: 15  
Number of pallets: 30  
  
Choose the algorithm:  
1. Backtracking  
2. Dynamic Programming  
3. Approximation Solution (Greedy Approach)  
4. Hybrid approach
```

Best Functionalities

- Throughout the development of the project, we truly appreciated some features that made the experience more enjoyable and productive, such as:
 - The interactive menu system, which allowed us to test and compare different solving strategies easily without having to modify the code manually every time;
 - The definition of the pallet structure, which made it intuitive to work with the items and improved the readability and modularity of our algorithms;
 - The execution time measurement, added to each solving method, which helped us evaluate performance in a concrete and visual way;
 - The reuse of the solver.cpp file across all algorithms, keeping the project organized and centralizing the core logic;

Main Difficulties

- However, there were also certain aspects where we faced some challenges:
 - Even though we understood the Knapsack Problem theoretically, it was not straightforward to implement the backtracking and dynamic programming solutions efficiently;
 - In the beginning, structuring the hybrid and greedy + local search approaches in a modular and clean way was tricky, as they combine ideas from multiple paradigms;
- **Work Distribution:**
 - Code Implementation: João, Afonso, Gonçalo;
 - Comments and Documentation: Afonso, Gonçalo;
 - Presentation: João.